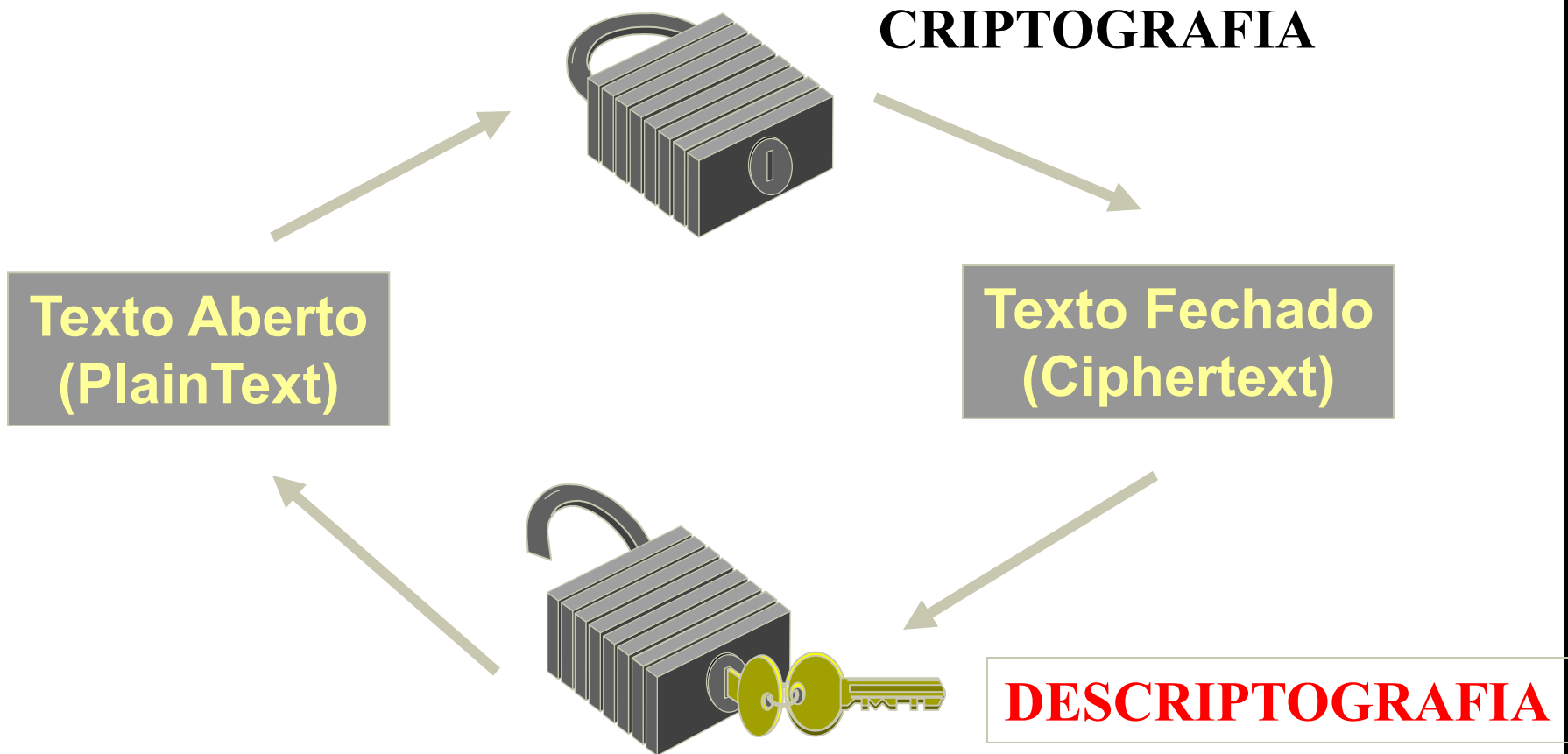


CRIPTOGRAFIA SIMETRICA E ASSIMÉTRICA

Professor

Edgard Jamhour

Criptografia e Descriptografia



Sistema de Criptografia Simples

Caesar Cipher: Shift Cipher

Substituição de letras pelas letras deslocadas de N.

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
↕ ↕
D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

**Nada de novo
no front.**

$N = 3$

**Qdgd gh qryr
qr iurqw.**

$N = 4$

**Rehe hi rszs rs
jvstx.**

Espaço das Chaves (KeySpace)

Uma chave é um valor específico do espaço de chaves (keyspace).

No exemplo anterior:

- Keyspace = 26
- $N = 3$, é a chave específica.

Algoritmos modernos:

- Chaves binárias: 128, 256, 1024, 2048 bits

Tipos de Criptografia:

- Simétrico: $\text{Keyspace} \cong 2^{\text{tamanho da chave}}$
- Assimétrico: $\text{Keyspace} \lll 2^{\text{tamanho da chave}}$

Cifra de Viginère

A	AB	CD	EF	GH	IK	LM	NO	PQ	RS	TU	XY	Z&
B	AC	BK	DU	EI	FL	GN	HO	MY	PS	QX	RT	Z&
C	AD	BG	CZ	EK	FM	HT	IX	LR	NP	OQ	S&	UY
D	AE	BZ	CT	DK	FI	GS	HY	LQ	MX	NR	O&	PU
E	AF	BL	CI	DH	EU	GK	MT	NQ	OR	P&	SX	YZ
F	AH	BF	CL	DG	EQ	IY	KP	MU	NS	O&	RX	TZ
G	AG	BI	CL	DN	ER	FP	HT	KU	M&	OX	QY	SZ
H	AI	BT	CS	DO	EL	F&	GH	KM	NQ	PR	UY	XZ
I	AK	BT	CS	DX	EI	FL	GZ	HY	M&	NP	OQ	RU
K	AL	BO	CP	DG	ER	FS	HU	IX	KY	MZ	N&	QT
L	AM	BZ	CD	EG	FI	HK	LN	OR	PS	QU	TY	X&
M	AN	BO	CP	DQ	ER	FS	GT	HU	IX	KY	LZ	M&
N	AO	BC	DM	EP	FS	GN	HY	IU	KT	LQ	R&	XZ
O	AP	BL	CK	DQ	ES	FU	GX	HZ	I&	MO	NR	TY
P	AQ	BX	CU	DZ	ES	FO	GY	HT	IN	KR	L&	MP
Q	AR	BZ	CT	DH	EU	FQ	GO	IL	KN	MP	SY	X&
R	AS	BN	CQ	DT	EU	FY	G&	HO	IP	KR	LX	MZ
S	AT	BP	CQ	DR	E&	FS	GU	HX	IY	KZ	LN	MO
T	AU	BY	CM	DX	E&	FH	GQ	IR	KZ	LS	NP	OT
U	AX	BL	CO	DQ	ES	FU	GT	HY	IN	KZ	M&	PR
X	AY	B&	CZ	DE	FX	GU	HI	KT	LS	MR	NP	OQ
Y	AZ	BU	CG	DH	EX	FY	IO	K&	LN	MP	QS	RT

S. Tomokiyo

Cipher between Marie-Antoinette and Fersen (1791-1792)

l e r o i e t l a r e i n e
D E P U I S D E P U I S D E
Q U K C E & C B Q P I Y R U

Plain Text = le roi et la reine

Esta cifra é denominada:
Cifra de Substituição Polialfabética

Essa é a variante usada por Maria Antoniette

Quiz 1

Qual o espaço de chaves do algoritmo utilizado por Maria Antoinette?

- A. O mesmo do Caesar Cipher, pois é baseado no alfabeto.
- B. É 2^{26} , pois o alfabeto tem 26 letras, e o algoritmo não diferencia maiúsculas de minúsculas
- C. Pode ser maior que 2^{26} .
- D. É exatamente 6^{22} .
- E. É igual ao número de palavras no dicionário Francês.

Quanto é 2^{128} ?

Massa do Planeta Terra em g:

- 5.972×10^{27}

Espaço de chaves da criptografia simétrica com 128 bits

- 3.4×10^{38}

Número de átomos no planeta Terra:

- 1×10^{50}

Valor numérico de 2^{128} em decimal:

- 340.000.000.000.000.000.000.000.000.000.000.000.000.000

Massa da Terra em gramas em decimal:

- 5.972.000.000.000.000.000.000.000.000

= 69 bilhões de vezes a massa da terra em gramas

Força Bruta

Força bruta é inviável para chaves de 128 bits

Mas ela foi usada para quebrar algoritmos de chaves menores no passado e 128 bits.

- DES (Data Encryption Standard) = Chaves de 56 bits criado em 1976
- Espaço de chaves = 72 quatrilhões

<u>Keyspace</u> = 2^{56}	Computador: Deep Crack (1856 chips) Custo = US\$ 250.000 em 1998 90 bilhões de chaves por segundo Tempo para encontrar uma chave: <u>56 horas</u>
<u>Keyspace</u> = 2^{128}	Computador: Hipotético 1 quintilhão de chaves por segundo Tempo para testar todas as chaves: <u>100 milhões de bilhões de anos.</u>



Análise de Frequência

15 (87 times)	12 (11 times)
7 (56 times)	6 (11 times)
22 (54 times)	4 (9 times)
24 (42 times)	29 (8 times)
21 (41 times)	9 (8 times)
23 (40 times)	3 (7 times)
25 (40 times)	19 (5 times)
5 (38 times)	0 (3 times)
18 (35 times)	1 (1 time)
2 (26 times)	17 (1 time)
14 (23 times)	32 (1 time)
26 (18 times)	33 (1 time)
11 (14 times)	40 (1 time)
13 (13 times)	61 (1 time)
8 (12 times)	84 (1 time)

New York Sept. 24. 1781

My Lord

I was honoured yesterday with your Lordships Letter of the 10th and 17th Instant. And so, 40. 6. 7. 39. 54. 7. 22. 7. 11. 15. 15. 22. 21. 5. 4. 24. 26. 22. 18. 15. 26. 2. 7. 4. 7. 6. 14. 4. 15. 5. 15. 25. 7. 2. 24. 26. 26. 21. 8. 15. 25. 23. 18. 15. 3. 14. 22. 18. 21. 20. 14. 7. 13. 21. 23. 21. 23. 14. 15. 22. 15. 25. 11. 21. 5. 15. 14. 22. 18. 7. 22. 7. 14. 24. 19. 15. 5. 0. 0. 0. 11. 15. 5. 25. 7. 5. 5. 7. 6. 14. 26. 21. 2. 15. 27. 12. 7. 2. 2. 14. 15. 15. 11. 12. 7. 25. 3. 15. 14. 24. 5. 12. 24. 7. 25. 14. 22. 15. 15. 21. 5. 4. 23. 23. 18. 21. 29. 23. 7. 5. 14. 22. 18. 15. 21. 24. 21. 5. 22. 15. 17. 13. 25. 22. 21. 24. 5. 23. 24. 26. 22. 18. 15. 5. 7. 14. 7. 5. 14. 7. 25. 11. 13. 11. 7. 14. 15. 21. 5. 7. 26. 15. 9. 14. 7. 13. 22. 22. 24. 25. 15. 2. 21. 15. 19. 15. 13. 22. 6. 7. 5. 14. 7. 26. 22. 15. 25. 9. 7. 25. 12. 23. 8. 24. 24. 20. 15. 25. 7. 22. 15. 9. 21. 22. 18. 18. 24. 6. The Fleet 2. 24. 5. 23. 21. 23. 22. 23. 23. 26. 23. 23. 7. 21. 2. 24. 26. 22. 18. 15. 2. 21. 5. 15. 22. 18. 25. 15. 15. 24. 26. 9. 18. 21. 3. 13. 7. 26. 15. 22. 18. 25. 15. 15. 14. 15. 8. 3. 15. 35. 23. 22. 18. 15. 25. 15. 21. 23. 15. 19. 16. 25. 13. 25. 15. 7. 24. 5. 21. 24. 18. 24. 29. 15. 9. 15. 23. 18. 7. 2. 2. 23. 22. 7. 25. 22. 26. 25. 24. 11. 18. 15. 5. 8. 15. 7. 12. 24. 6. 22. 22. 18. 15. 5. 24. 8. 22. 24. 12. 15. 25. I have the Honour to be

Your Lordships
Most obedient and most
humble Servant.

Clinton to Cornwallis, 24 September 1781
Clements Library, University of Michigan

Análise de Frequência

Letras mais usadas em inglês: **ETAONISH**

- 15=E

Palavra mais usada em inglês: **THE**

A sequência: 22 18 15 é a que se repete mais vezes (12)

- 22=T, 18=H e 15=E

Outras deduções:

T H 25 E E = **THREE**; T H 25 E = **THERE**

- 25 = R

Analisando a sequência:

"a 26 t e r = 9 a r 14 23 - 8 24 24 29 e r a t e - 9 21 t h - y 24 6"

"a **F** t e r = **W** a r **D S C O O P** e r a t e **W I** t h y **O U**

Criptografia Simétrica e Assimétrica

Dois sistemas de criptografia são usados atualmente:

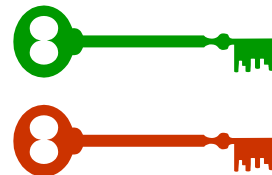
Sistemas de chave secreta (**secret-key**)



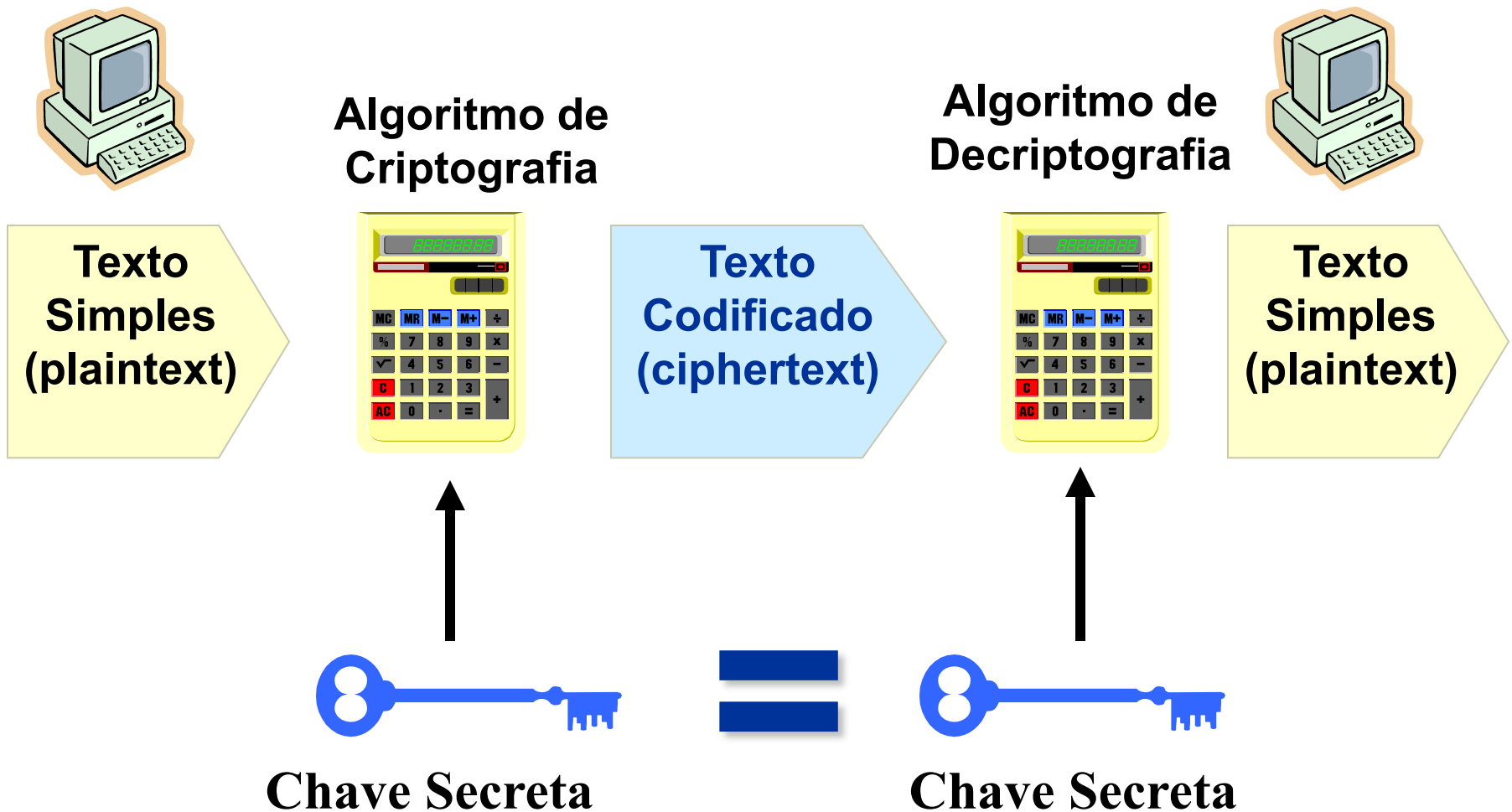
- Também denominados simétricos
- Trabalha com uma única chave, denominada SECRETA.

Sistemas de chave pública (**public-key**)

- Também denominado assimétrico
- Trabalho com um par de chaves
 - CHAVE PÚBLICA
 - CHAVE PRIVADA



Chave Secreta (Criptografia Simétrica)



Criptografia Simétrica e Chave de Sessão

A **chave de simétrica** ou **secreta** não é sempre a mesma

A troca de chaves é feita em cada nova **sessão**

A troca de chaves pode ser feita durante uma sessão usando como condição para a troca o tempo de uso de uma mesma chave ou o volume de tráfego já criptografado

Reutilizar uma chave ou usar sempre a mesma chave é considerado uma **falha de segurança**

- Se uma chave for descoberta, tudo que foi que foi feita por ela fica desprotegido.

Como a chave simétrica muda a cada sessão, ela é as vezes referenciada como **chave de sessão**

Quiz 2

Uma boa prática em criptografia simétrica consiste em usar uma chave secreta aleatória denominada **chave de sessão**. Para cada nova sessão uma nova chave é usada. Quando começa uma nova sessão? Indique todas as corretas.

- A. Quando o usuário acessar um computador diferente.
- B. Quando o usuário acessar uma aplicação diferente.
- C. Cada vez que o usuário se autenticar.
- D. Automaticamente, após um certo intervalo de tempo.
- E. Automaticamente, após um certo volume de dados.

Block Ciphers vs Stream Ciphers

Block Ciphers dividem a mensagem em blocos de tamanhos iguais

- 64-bits ou 128-bits

As operações de criptografia são feitas sobre os blocos e o resultado pode levar em conta também o resultado da criptografia de outros blocos

Stream Ciphers criptografam a mensagem como um fluxo contínuo de bits ou bytes

- Dados enviados em fluxo contínuo, como Video ou Audio.
- Precisa garantir que a chave de criptografia nunca seja reusada

Block Ciphers são mais eficientes quando o tamanho da mensagem a ser criptografada é previamente conhecido

XOR Cipher = Stream Cipher

Algoritmos do tipo stream utilizam frequentemente uma simples operação lógica de XOR para criptografar os dados.

Esses algoritmos são chamados de ciphers aditivos, e tem as seguintes propriedades:

$$A \oplus 0 = A,$$

$$A \oplus A = 0,$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C),$$

$$(B \oplus A) \oplus A = B \oplus 0 = B,$$

Exercício:

Utilizando o Shell do Python, execute as seguintes operações e anote o resultado:

1) $65 \wedge 0$

2) $65 \wedge 65$

3) $65 \wedge 64$

4) $65 \wedge 64 \wedge 97$

5) $97 \wedge 64 \wedge 65$

6) **$65 \wedge 64 \wedge 64$**

7) **$65 \wedge 64 \wedge 65$**

8) $65 \wedge 64 \wedge 97 \wedge 64$

9) $65 \wedge 97$

Quiz 3

Em relação ao exercício anterior, se 'A' for o dado e '@' for a chave, o que significam as operações 6 e 7?

- A. São exemplos de dupla criptografia.
- B. A operação 6 é descriptografia e a operação 7 é dupla criptografia.
- C. Ambas as operações não fazem sentido, pois o XOR é sempre entre dois elementos apenas.
- D. A operação 6 é descriptografia e a operação 7 mostra como é simples descobrir a chave de criptografia.

Exemplo:

"Wiki" (01010111 01101001 01101011 01101001 in 8-bit ASCII)

Suponha que foi escolhida a chave: 11110011

Criptografia:

$$\begin{array}{r} 01010111 \ 01101001 \ 01101011 \ 01101001 \\ \oplus \ 11110011 \ 11110011 \ 11110011 \ 11110011 \\ \hline = \ 10100100 \ 10011010 \ 10011000 \ 10011010 \end{array}$$

Descriptografia:

$$\begin{array}{r} 10100100 \ 10011010 \ 10011000 \ 10011010 \\ \oplus \ 11110011 \ 11110011 \ 11110011 \ 11110011 \\ \hline = \ 01010111 \ 01101001 \ 01101011 \ 01101001 \end{array}$$

BASE64

[Base64 Decode and Encode - Online](#)

Um texto criptografado (ciphertext) pode incluir caracteres de controle e caracteres não imprimíveis.

Isso impede que ciphertext seja salvo em um arquivo ou em um banco de dados.

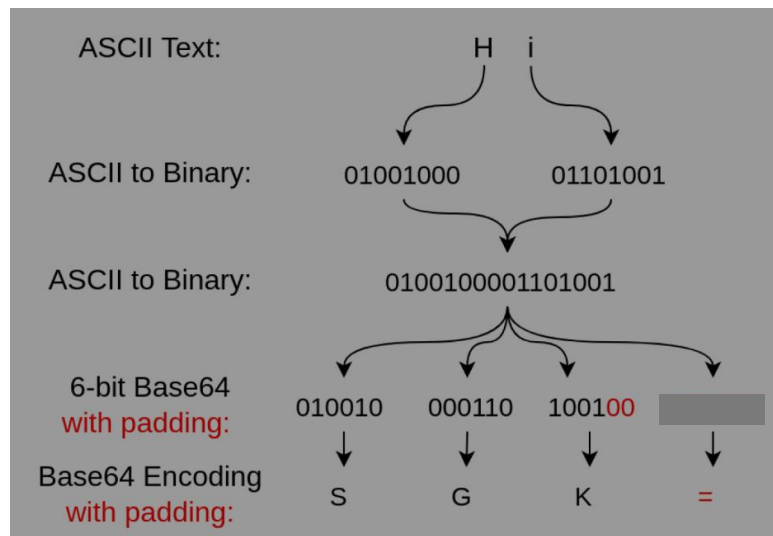
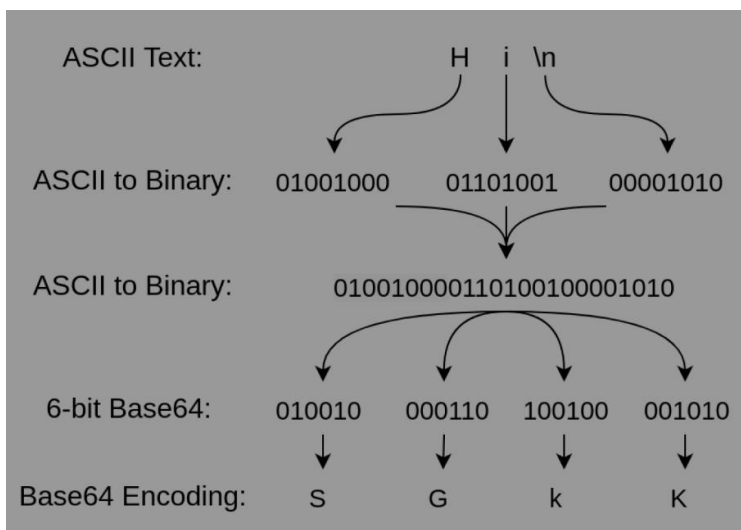
Um ciphertext codificado em base64 contém apenas caracteres imprimíveis ASCII conforme a tabela ao lado.

Base64 divide o texto em grupos de 6 bits (pois $2^6 = 64$) ao invés de 8.

BASE64 INDEX TABLE

0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

BASE64



$$010010 = 16 + 2 = 18$$

$$000110 = 4 + 2 = 6$$

$$100100 = 32 + 4 = 36$$

$$001010 = 8 + 2 = 10$$

Essa codificação precisa que a mensagem seja múltiplo de 6 bits.

Mensagens que não são múltiplos de 6 bits recebem caracteres de padding (enchimento) no final.

Cada símbolo (=) indica que 2 bits (00) foram inseridos.

Problemas com o XOR Cipher: Colisão do PlainText e CipherText

Suponha que duas mensagens A e B sejam criptografadas com a mesma chave X:

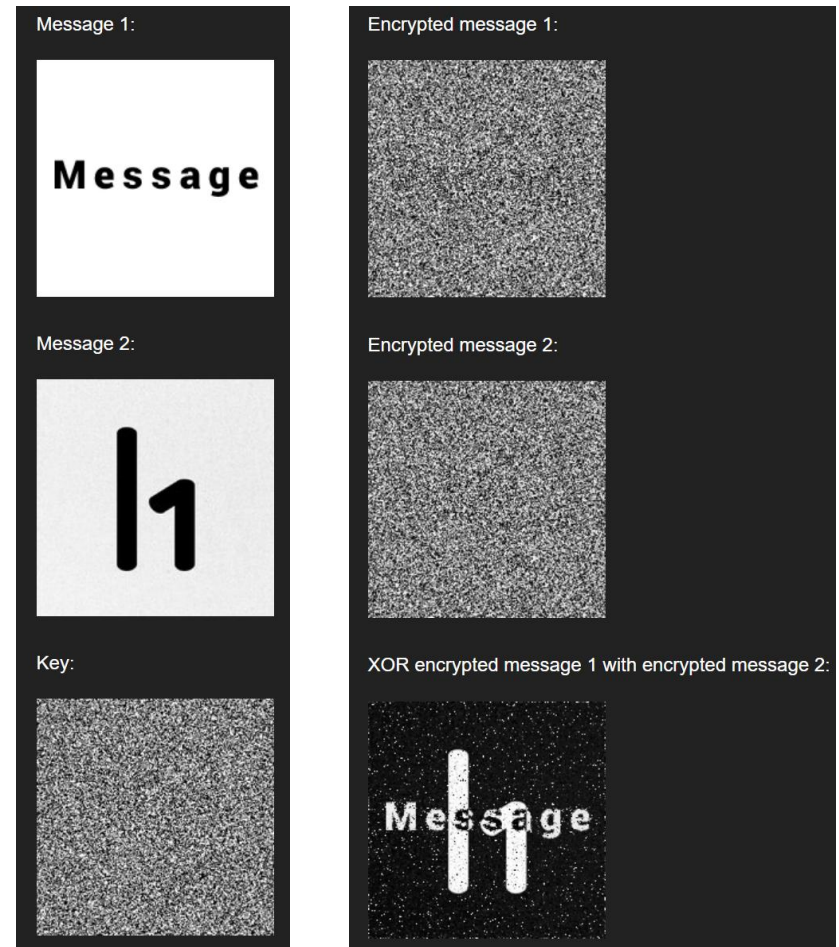
- $E(A) = A \text{ XOR } X$
- $E(B) = B \text{ XOR } X$

De acordo com as propriedades do XOR a seguinte relação é equivalente:

- $E(A) \text{ XOR } E(B) = A \text{ XOR } B$

Veja as consequências desse problema em [exemplo1](#) e [exemplo2](#).

A maneira de se proteger é nunca repetir a chave. Isso é feito combinando a chave com um outro valor na forma de um contador, por exemplo.



Exercício

Utilize as funções em Python fornecidas para fazer a verificação da propriedade:

$A \text{ XOR } B = E(A) \text{ XOR } E(B)$
e suas consequências.

```
def xor_cipher(plain, key):
```

```
    hkey = []  
    result = []
```

```
    for i in key:
```

```
        hkey.append(ord(i))
```

```
    i = 0
```

```
    for c in plain:
```

```
        result.append(ord(c)^hkey[i])
```

```
        i += 1
```

```
        if i == len(key):
```

```
            i = 0
```

```
    return result
```

```
def to_ascii(result):
```

```
    cipher = ""
```

```
    for i in result:
```

```
        cipher += chr(i)
```

```
    return cipher
```

Problemas com o XOR Cipher: Tampering

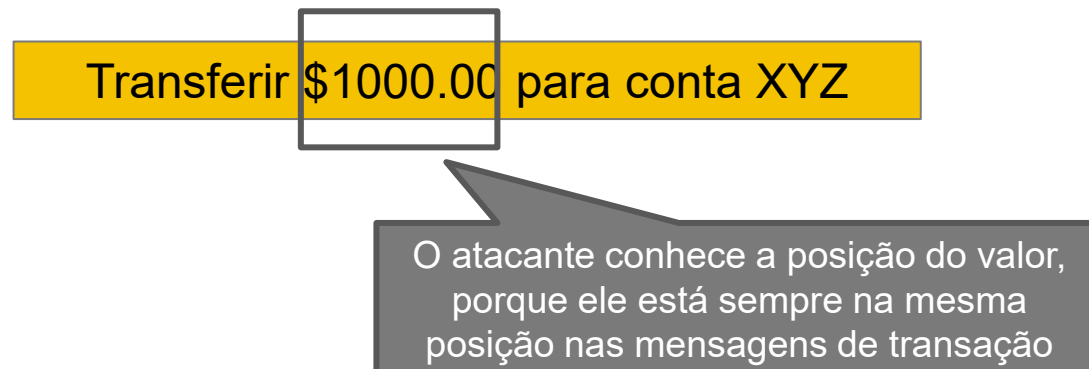
Ataque BIT FLIPPING: falsificar partes da mensagem sem conhecer a chave.

- Também definido com Replay e Man-in-The-Middle

Suponha que a comunicação entre Alice e Bob foi interceptada, mas está criptografada com a chave $C(K)$

- CHAVE xor Transferir \$1000.00 para conta XYZ

Caso o atacante conheça a posição do valor \$1000.00, ele pode alterar o valor sem conhecer a chave



Problemas com o XOR Cipher: BIT FLIPPING e ICV

A alteração pode ser feita com o seguinte procedimento:

- $(\text{CHAVE} \text{ xor } "\$1000.00") \text{ xor } ("\$1000.00" \text{ xor } "\$9500.00") =$
- $\text{CHAVE} \text{ xor } "\$1000.00" \text{ xor } "\$1000.00" \text{ xor } "\$9500.00" =$
- $C(K) \text{ xor } "\$9500.00"$

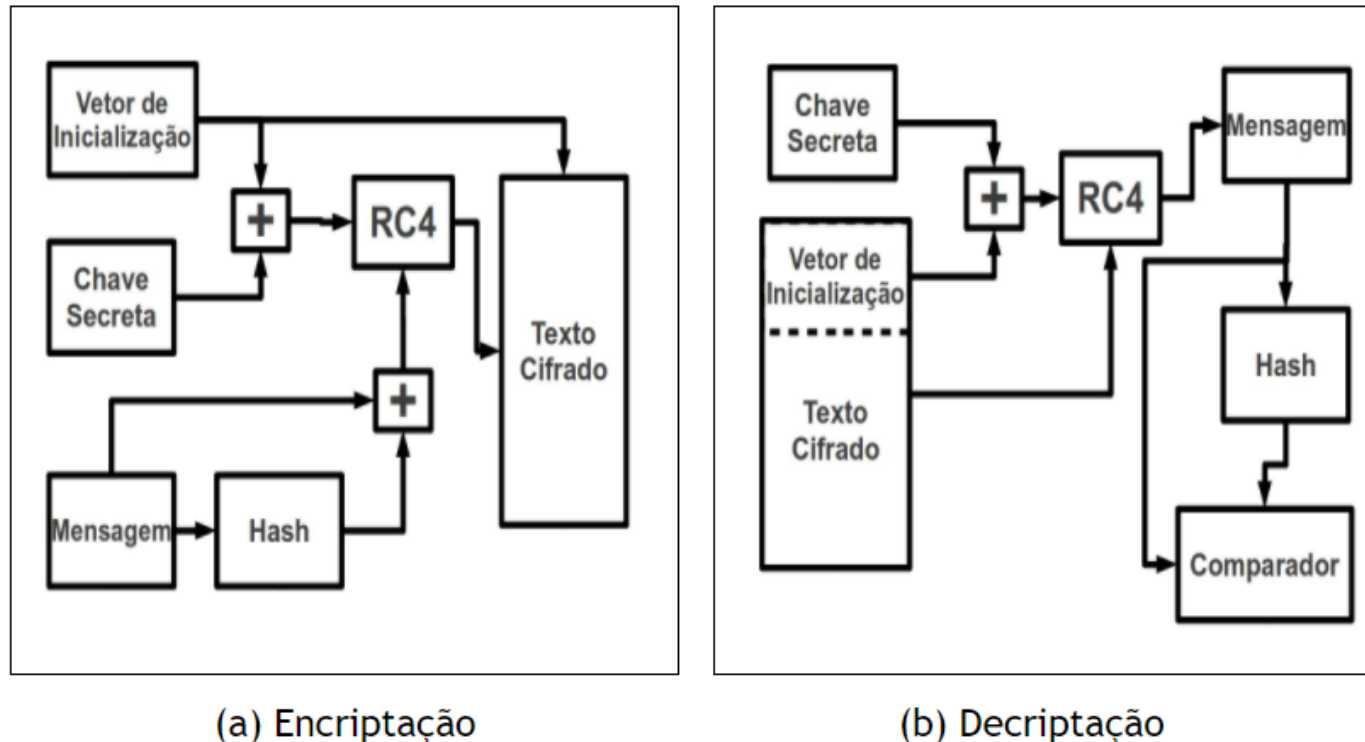
A maneira de proteger-se contra esse ataque é incluir um código de verificação de integridade na mensagem.

Esse código é denominado MAC ou ICV:

- MAC: Message Authentication Code
- ICV: Integrity Check Value

Vetor de Inicialização: IV

Uma maneira de contornar o problema é evitar que a mesma chave seja reutilizada utilizando um vetor de inicialização transmitido em PlainText junto com a mensagem.



IV de 24 bits concatenado com a chave

Figura 2: Diagrama de Blocos do Protocolo WEP

Pseudo-Aleatório

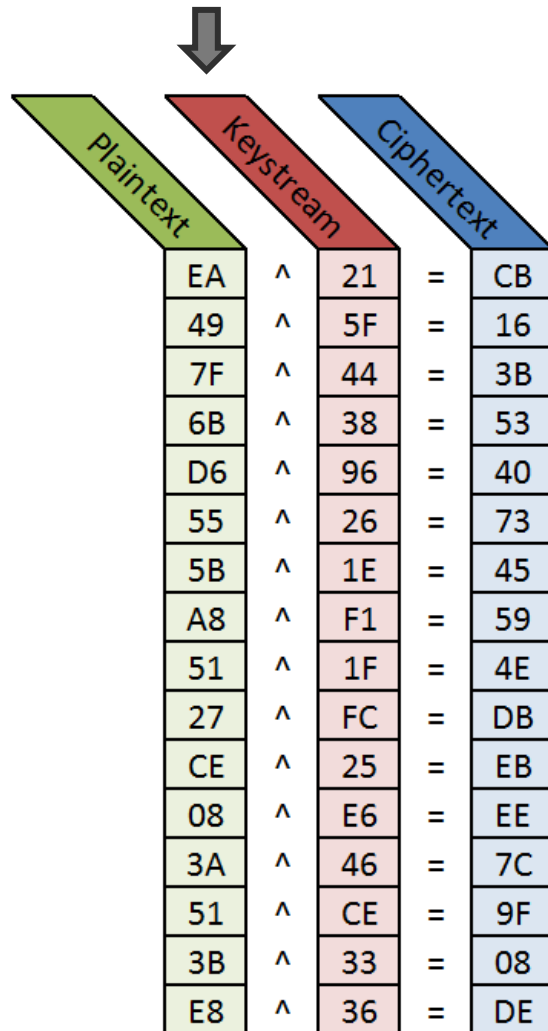
Pseudoaleatório é algo que **parece aleatório**, mas foi gerado por um **algoritmo determinístico**. Isso significa que, se o algoritmo começar com a mesma informação inicial, ele produzirá exatamente a mesma sequência.

Em uma cifra de fluxo como o RC4, o que não pode acontecer é reutilizar a **mesma sequência pseudoaleatória**. Uma maneira de evitar isso seria trocar a **chave** a cada novo fluxo de dados. O problema é que combinar uma nova chave entre emissor e receptor tem custo alto.

Por isso existe o **IV**. Ele permite variar a entrada do algoritmo sem precisar trocar a chave principal o tempo todo. Assim, a **mesma chave pode ser mantida por mais tempo**, enquanto o **IV muda a cada fluxo**, fazendo o algoritmo gerar uma nova sequência pseudoaleatória.

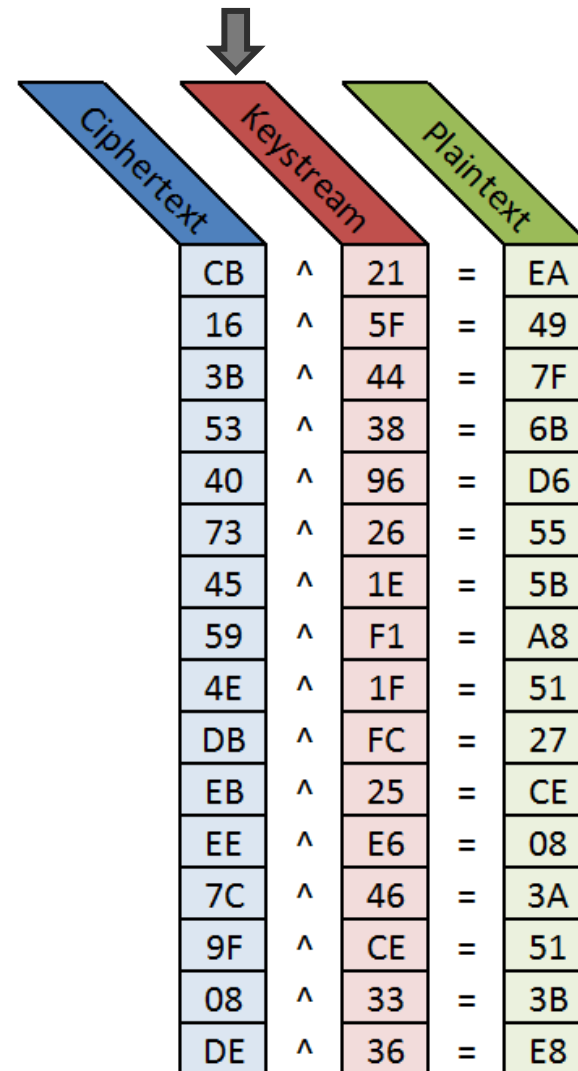
RCA e Pseudo-Random KeyStream

CHAVE combinada com IV



Plaintext		Keystream		Ciphertext
EA	^	21	=	CB
49	^	5F	=	16
7F	^	44	=	3B
6B	^	38	=	53
D6	^	96	=	40
55	^	26	=	73
5B	^	1E	=	45
A8	^	F1	=	59
51	^	1F	=	4E
27	^	FC	=	DB
CE	^	25	=	EB
08	^	E6	=	EE
3A	^	46	=	7C
51	^	CE	=	9F
3B	^	33	=	08
E8	^	36	=	DE

CHAVE combinada com IV



Ciphertext		Keystream		Plaintext
CB	^	21	=	EA
16	^	5F	=	49
3B	^	44	=	7F
53	^	38	=	6B
40	^	96	=	D6
73	^	26	=	55
45	^	1E	=	5B
59	^	F1	=	A8
4E	^	1F	=	51
DB	^	FC	=	27
EB	^	25	=	CE
EE	^	E6	=	08
7C	^	46	=	3A
9F	^	CE	=	51
08	^	33	=	3B
DE	^	36	=	E8

Confusion and Diffusion

Confusion significa que cada bit da chave afeta vários bits do texto cifrado.

- Esconde o relacionamento entre a chave e a mensagem criptografada
- Torna difícil encontrar a chave por aproximações sucessivas

Diffusion significa que cada bit da mensagem original afeta vários bits do texto cifrado.

- Esconde o relacionamento entre o plaintext e o ciphertext
- Torna difícil decodificar a mensagem por análise de padrões.

A qualidade de um algoritmo de criptografia é fortemente dependente dessas duas características.

Como saber?

Confusion

Se você criptografar a mesma mensagem com uma chave levemente diferente, o resultado muda completamente.

- Mesma mensagem
- Chaves diferentes

Diffusion

Se você criptografar um mensagem levemente diferente com a mesma chave, o resultado muda completamente.

- Mensagens diferentes
- Mesma chave

Quiz 4

Quais propriedades o algoritmo de XOR cipher simples, gerando o keystream por repetição da chave e sem vetor de inicialização, satisfaz?

- A. Apenas Confusion.
- B. Apenas Difusion.
- C. Confusion e Difusion.
- D. Nenhuma das duas.
- E. Depende do tamanho da chave.

Quiz 5

Utilize o código anterior verifique quais propriedades o algoritmo RC4 satisfaz?

- A. Apenas Confusion.
- B. Apenas Difusion.
- C. Confusion e Difusion.
- D. Nenhuma das duas.
- E. Depende do tamanho da chave.

Problema do **STREAM CIPHER**

Observe que é difícil (ou IMPOSSÍVEL) obter a propriedade de difusão se um byte é criptografado de cada vez.

Para conseguir difusão é preciso conhecer o conjunto de bytes que precisa ser criptografado antes de iniciar a criptografia.

Na prática isso limitaria muito o uso da criptografia na transmissão de dados.

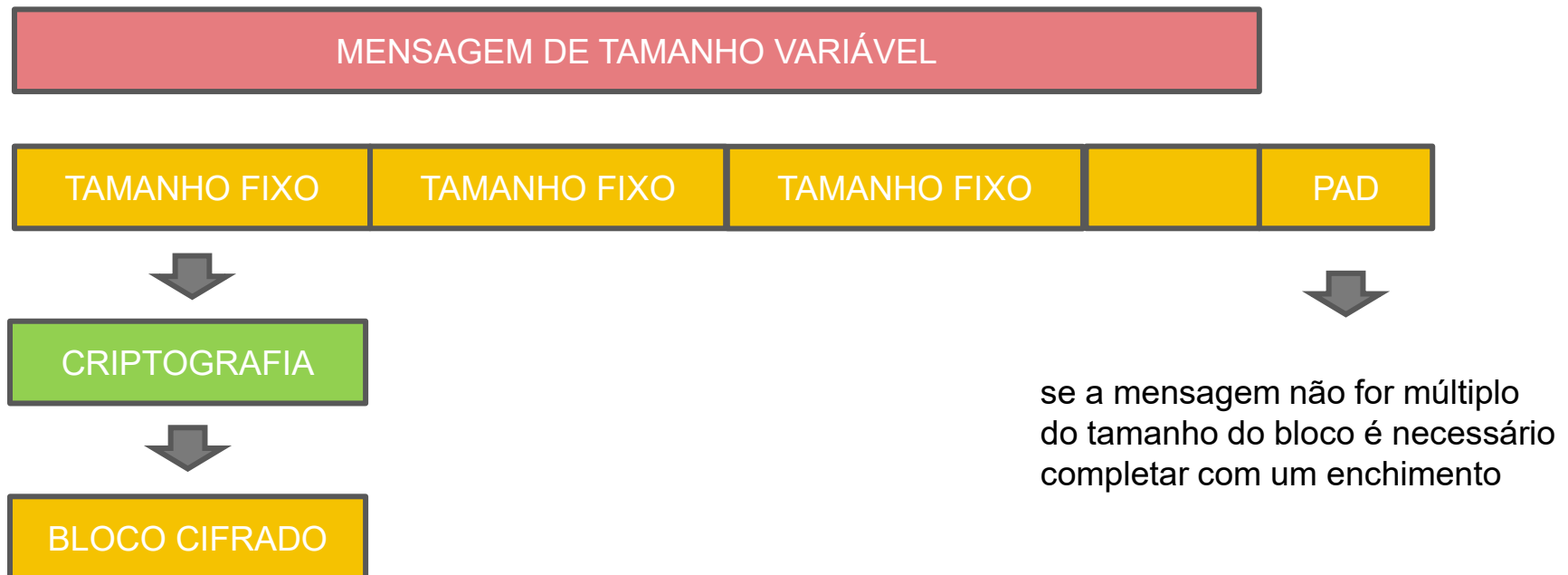
Então optou-se por uma solução intermediária;

DECOMPOR A MENSAGEM EM BLOCOS

Block Ciphers

Permite melhorar a segurança através de operações simples de permuta e substituição aplicadas sucessivas vezes sobre um conjunto de bytes fixos.

- Alta difusão
- Maior imunidade a inserção de símbolos no plaintext sem detecção



Difusão no Block Cipher

Se um único bit for alterado, ele precisa mudar completamente o resultado da criptografia.

Dois blocos com apenas este bit diferente precisam gerar blocos criptografados com muitos bits diferentes.



ABCDEFGH → çǻǻ£ç®¬ª

BBCDEFGH → ÁßÜØ»¶A*

E a maneira como isso acontece precisa depender da chave

E precisa parecer aleatória, para evitar ciptoanálise diferencial

CipherModes (Block Cipher Mode of Operation)

ECB: Electronic Codebook Mode

- criptografa os blocos de forma independente
- considerado inseguro porque mantém padrões

CBC: Cipher Block Chaining

- faz um xor com o bloco anterior antes de criptografar
- Pipeline Delays: não pode ser processado em paralelo

CTR: Counter Mode

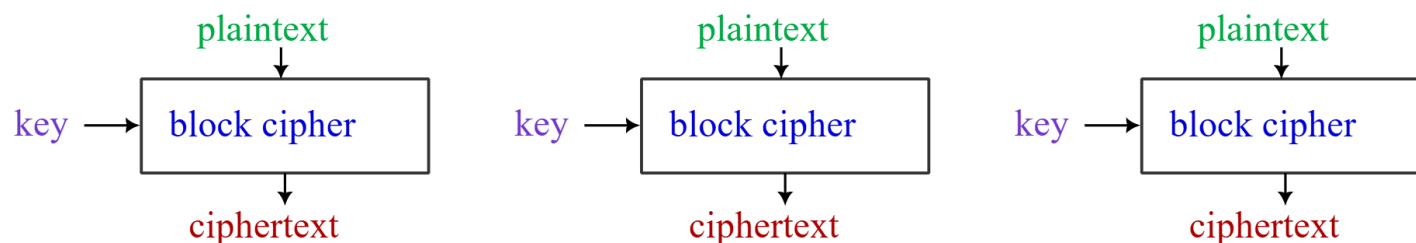
- usa a cifra de blocos para gerar um keystream criptografando um contador
- pode ser processado em paralelo mas pode sofrer bit flipping

Galois/Counter Mode (GCM)

- semelhante ao CTR mas adiciona uma tag de autenticação para identificar bit-flipping

MODO ECB: Eletronic CodeBook

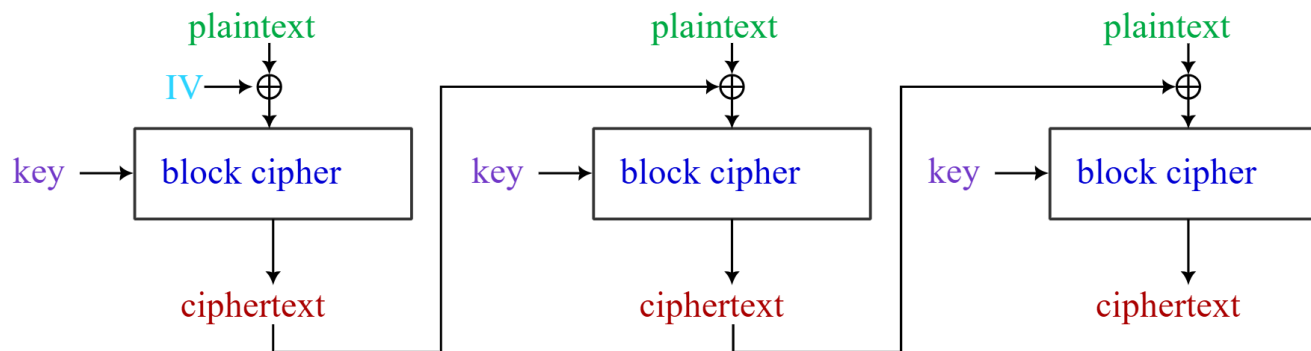
Electronic codebook (ECB)



O ECB gera **blocos cifrados iguais** se os blocos plain forem iguais. Isso só é aceitável em mensagens muito pequenas se a chave for trocada a cada mensagem; mas isso é uma solução **pouco prática** e por isso **quase não é usada**.

MODO CBC: Cipher Block Chaining

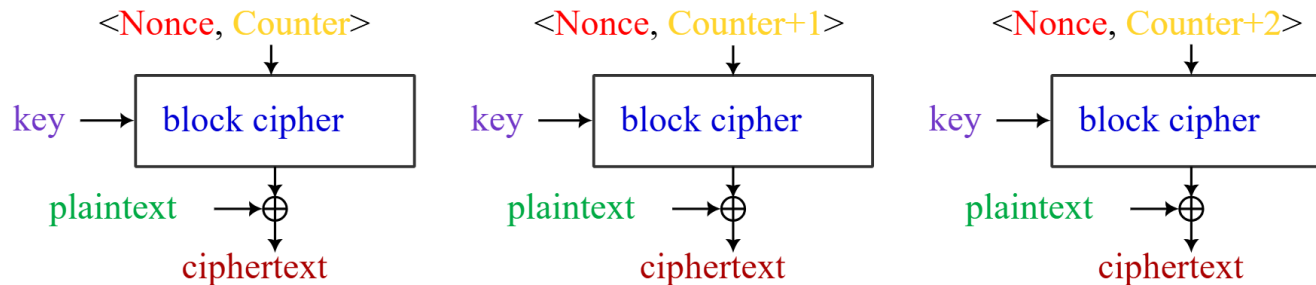
Cipher block chaining (CBC)



Cada bloco passa por **dupla cifragem**: primeiro sofre uma **cifragem por XOR**, usando o **IV** no primeiro bloco ou o **bloco cifrado anterior** nos demais, e depois o resultado é enviado para a **cifra de blocos**;

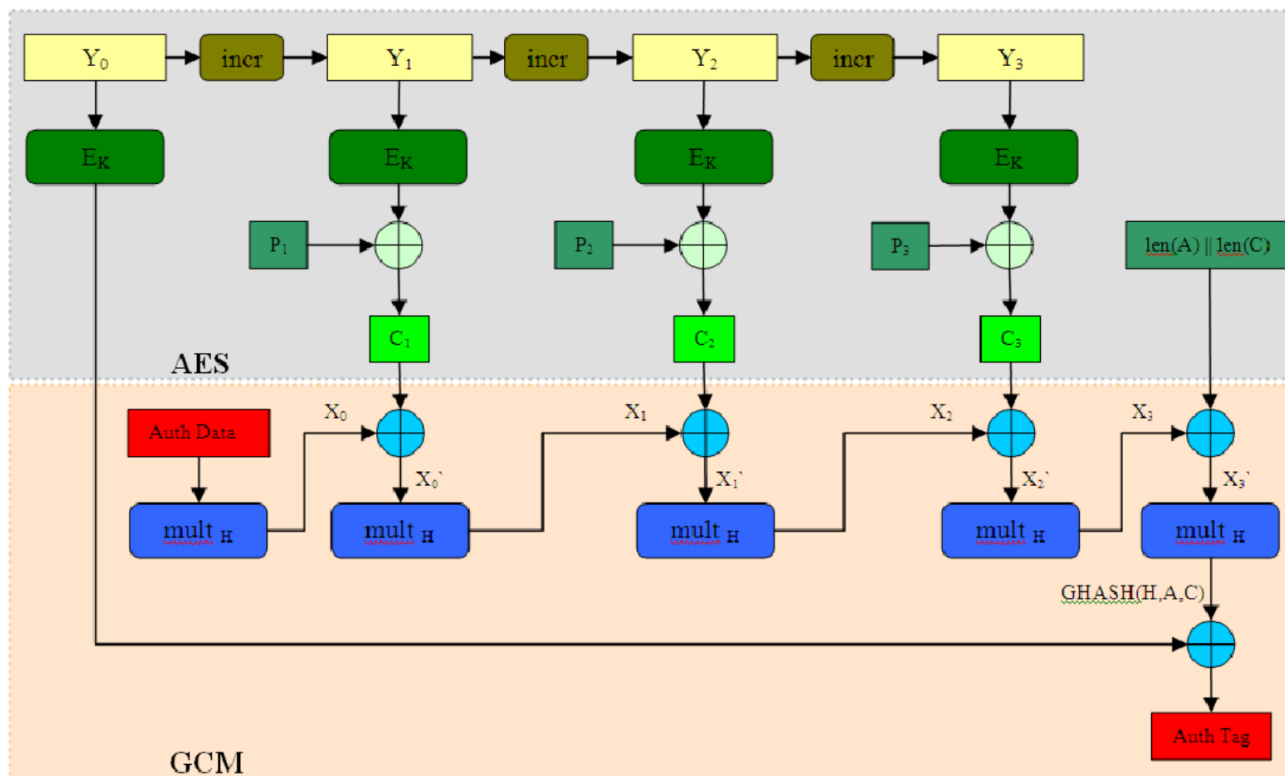
MODOS CTR

Counter (CTR)



No modo **CTR**, a cifra de blocos gera o **keystream** cifrando um contador, funcionando na prática como uma **cifra de fluxo**; o problema é que isso a torna vulnerável a **bit flipping**, pois um atacante pode alterar bits específicos do texto cifrado e provocar mudanças previsíveis e locais no texto claro, sem precisar decifrá-lo.

MODO GCM



No modo **GCM**, além de cifrar, o algoritmo calcula uma **tag de autenticação** combinando matematicamente **todos os blocos do texto cifrado**, o **nonce/IV** e, opcionalmente, os **dados autenticados adicionais (AAD)**; assim, se qualquer bit for alterado, a tag calculada na recepção não confere, detectando **bit flipping** e outras modificações.

Algoritmos Simétricos (Quadro Resumo)

Algoritmo	Tipo	Método	Tamanho de chave	Criação	Retirada
AES	Criptografia simétrica	Cifra de blocos de 128 bits	128, 192 ou 256 bits	2001	—
ChaCha20	Criptografia simétrica	Stream cipher	256 bits	2008	—
3DES	Criptografia simétrica	Cifra de blocos de 64 bits	112 ou 168 bits	1978	2024
RC4	Criptografia simétrica	Stream cipher	variável	1987	2015*

O **AES** foi criado para substituir o DES por meio de uma competição pública organizada pelo NIST; o algoritmo escolhido foi o **Rijndael**, padronizado como AES em 2001. Ele acabou se tornando o padrão dominante também porque ganhou forte **aceleração em hardware** nos processadores modernos.

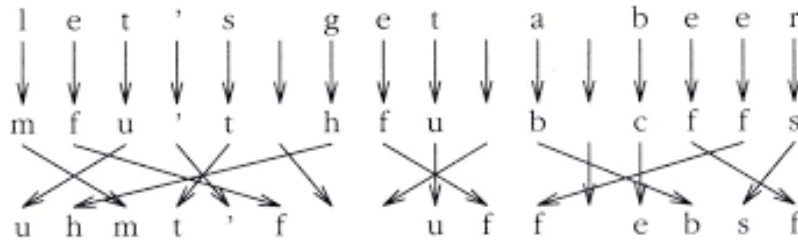
O **ChaCha20** foi criado por **Daniel J. Bernstein** com foco em ser uma cifra moderna, simples e muito eficiente em **software**. Diferentemente do AES, ele não depende de instruções dedicadas para ter bom desempenho, o que ajudou sua adoção em sistemas onde a aceleração de hardware para AES não disponível, como sistemas antigos e sistemas embarcados.

AES vs CHACHA

O **AES** usa operações como **substituição de bytes**, **permutação/deslocamento**, **mistura linear das colunas** e **XOR com subchaves**. O número de rodadas depende do tamanho da chave: **10 rounds** para 128 bits, **12 rounds** para 192 bits e **14 rounds** para 256 bits.

O **ChaCha20** usa apenas três operações simples: **adição modular**, **rotação** e **XOR**. Seu processamento é organizado em **20 rounds**, normalmente vistos como **10 double rounds**. Então, de forma didática: o **AES** usa transformações mais estruturadas e heterogêneas em menos rounds; o **ChaCha20** usa poucas operações muito simples, repetidas ao longo de mais rounds.

AES: Redes de Substituição e Permutação

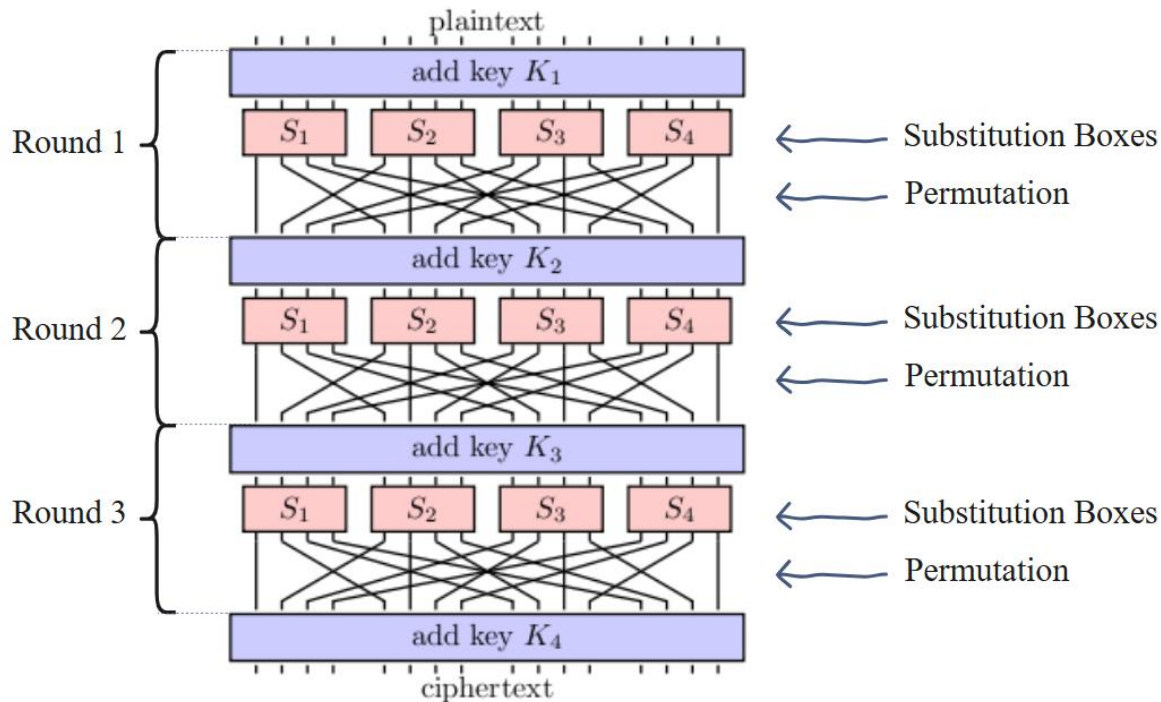


substitution

transposition

S-BOX

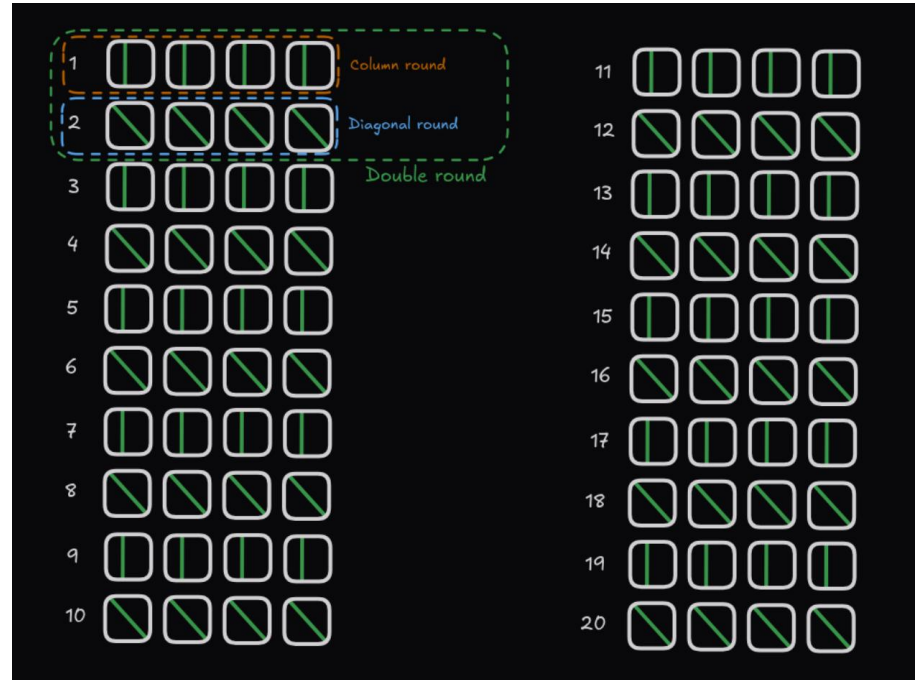
		0	1	2	3	...
31	0	63	7c	77	7b	
	1	ca	82	c9	7d	
	2	b7	fd	93	26	
	3	04	c7	23	c3	
	...					



CHACHA: gera blocos de Keystream

Estado inicial

Cons	Cons	Cons	Cons
Key	Key	Key	Key
Key	Key	Key	Key
Counter	Nonce	Nonce	Nonce



Para cada valor de um **contador**, o ChaCha20 monta um estado inicial com **constantes, chave, contador e nonce**, executa os **20 rounds**, soma o resultado ao estado inicial e produz **64 bytes de keystream**. Depois, para o próximo bloco, ele muda o **contador** e repete o processo.

Cada **round** do ChaCha20 mistura os valores da matriz interna usando apenas **adição modular, XOR e rotação**. Ele não gera sozinho um bloco novo; apenas embaralha mais o mesmo estado. Metade dos rounds atua sobre as **colunas** da matriz e a outra metade sobre as **diagonais**, para espalhar a influência de cada palavra por todo o bloco.

Quiz 6

Porque se usa múltiplos rounds no CHACHA e no AES?

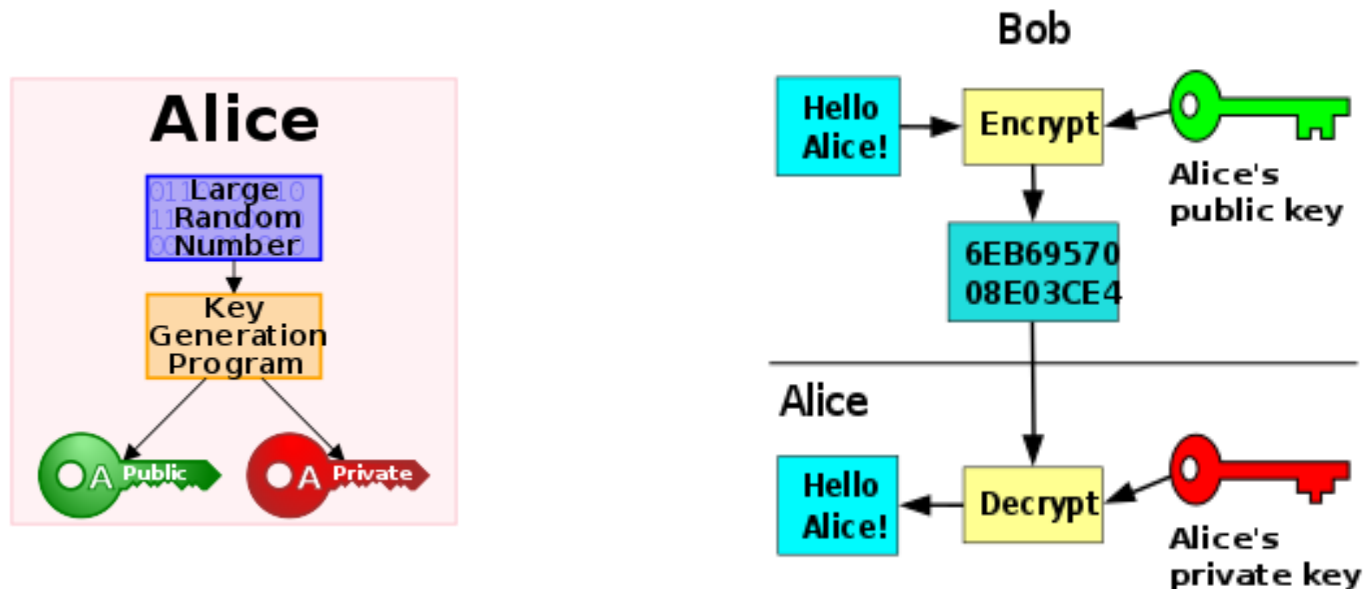
- A. Para tornar o algoritmo mais rápido.
- B. Para tornar o algoritmo mais simples.
- C. Para obter confusão e difusão.
- D. Para dificultar o uso de força bruta.

Chave Pública = CRIPTOGRAFIA ASSIMÉTRICA

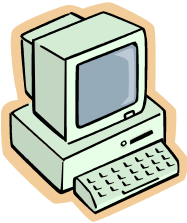
Sistema de Criptografia Assimétrica = PAR DE CHAVES

- A chave publica (não é secreta) criptografa a mensagem.
- A chave privada (é secreta) descriptografa a mensagem.

A chave pública deve ser distribuída para os transmissores para garantir comunicação segura com o receptor.

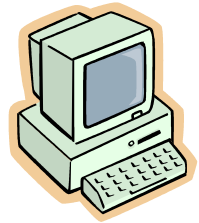


Chave Pública (Criptografia Assimétrica)

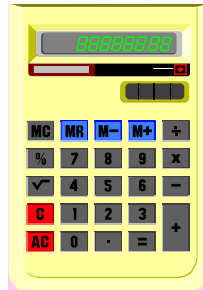


**Algoritmo de
Criptografia**

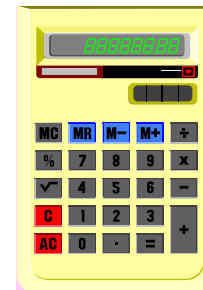
**Algoritmo de
Descriptografia**



**Texto
Simples
(plaintext)**



**Texto
Codificado
(ciphertext)**



**Texto
Simples
(plaintext)**



Chave Pública



Chave Privada

TRAPDOOR FUNCTION

Criptografia de chaves públicas precisam encontrar funções do tipo TRAPDOOR

Essas funções são fáceis de calcular em uma direção, mas muito difíceis no sentido reverso

Exemplo:

- Dado x , $y = f(x)$ é fácil de calcular
- Mas dado y , $x = f^{-1}(y)$ é um problema muito complexo

Nas aplicações de criptografia:

- X = Chave Privada
- Y = Chave Pública



Algoritmos de Chave Pública

RSA: Rivest–Shamir–Adleman

- Princípio: fatoração de números primos
- Patente: 1977

ECC: Elliptic-Curve Cryptography

- Princípio: curvas elípticas sobre campos finitos
- Conhecido a muito tempo, mas explorado a partir de 2015

Diffie-Hellman Key Exchange

- Princípio: cria um segredo compartilhado usando chaves públicas
- Publicado pelos autores em 1977, logo após o RSA

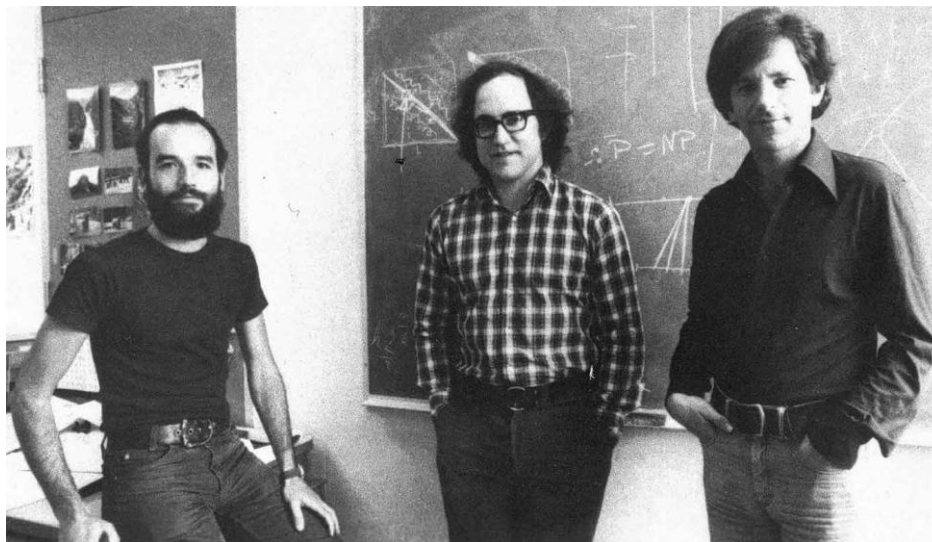
RSA e ECC são usados principalmente para **autenticação** por meio de **assinaturas digitais**. Já o Diffie-Hellman (DH) é usado para **combinar um segredo compartilhado** entre duas partes. O problema é que o DH, sozinho, **não autentica** quem está do outro lado. Por isso, ele normalmente é combinado com **RSA** ou **ECC**, que permitem verificar a identidade do peer e garantir que o segredo foi estabelecido com a parte correta, e não com um atacante no meio da comunicação.

RSA: (Rivest, Shamir, Adleman)

Ron Rivest, Adi Shamir, and Leonard Adleman publicaram o algoritmo pela primeira vez em 1978.

O mesmo algoritmo era conhecido pelo governo britânico desde 1973 (Clifford Cocks), mas só foi tornado público em 1997.

A RSA é um algoritmo muito lento quando comparado aos algoritmos simétricos. Por isso seu uso de criptografia é restrito a combinar um segredo ou validar uma assinatura digital.



RSA

Sejam **p**, **q** números primos (> 512 bits).

- $n = p \cdot q$
- escolher **e co-primo** com n tal que: $1 < e < (p-1)(q-1)$
- encontrar **d** tal que: $e \cdot d \% (p-1)(q-1) = 1$

n é o valor máximo do dado criptografado

As chaves são definidas da seguinte maneira:

- Chave pública: (n, e)
- Chave privada: (n, d)

Para criptografar uma mensagem “**m**” efetua-se a operação:

- $s = m^e \bmod n$

mod n faz com que o resultado seja sempre menor que n

Para descriptografar, efetua-se a operação:

- $m = s^d \bmod n$

RSA Algorithm Example

1. Escolher: $p = 3$ e $q = 11$
2. Calcular: $n = p * q = 3 * 11 = 33$
3. Calcular: $\varphi = (p - 1) * (q - 1) = 2 * 10 = 20$
4. Escolher: e tal que $1 < e < \varphi$ e e e n são co-primos
5. Calcular: d tal que $(d * e) \% \varphi(n) = 1$
6. Escolher: chave pública = $(e, n) \Rightarrow (7, 33)$
7. Escolher: chave privada = $(d, n) \Rightarrow (3, 33)$

solução possível: $e = 7$

solução possível: $d = 3$
pois $(3 * 7) \% 20 = 1$

8. Criptografar: $c = m^e \bmod n$: se $m = 2 \Rightarrow$

cipher text: $c = 2^7 \% 33 = 29$

9. Descriptografar: $m = s^d \bmod n$: se $c = 29 \Rightarrow$

plain text: $m = 29^3 \% 33 = 2$

Fatorar um Inteiro em Primos

No RSA, fatorar $n = p \times q$ significa descobrir os primos p e q a partir apenas de n . A dificuldade é que, mesmo sabendo que os fatores são primos e devem estar perto de $\sqrt{n}$, ainda há candidatos demais, e não se conhece um método direto e eficiente para encontrar exatamente o par correto.



[RSA Factoring Challenge - Wikipedia](#)

Por que mod n ($\% n$)?

A operação **mod** n é o que faz o RSA trabalhar sempre dentro do conjunto de valores definido pela chave, onde $n = p \times q$. Por isso, o RSA **não cifra mensagens de tamanho arbitrário**: ele cifra apenas **números menores que n** . Isso impõe um limite ao tamanho máximo da mensagem que pode entrar diretamente em uma operação de RSA.

Além disso, o resultado da cifragem também passa a ser representado dentro do espaço numérico de n , o que na prática significa que o texto cifrado tem um tamanho ligado ao tamanho da chave. Assim, no RSA, a chave não define apenas a segurança: ela também define **o tamanho máximo do dado que pode ser cifrado diretamente e o tamanho do valor cifrado produzido**.

Equivalencia de Espaço de Chaves

O RSA está condenado porque o tamanho da chave precisará aumentar muito para acompanhar o nível de segurança necessário para os próximos anos.

Symmetric Key Size (bits)	RSA and Diffie-Hellman Key Size (bits)	Elliptic Curve Key Size (bits)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	521

2030

TOP
SECRET

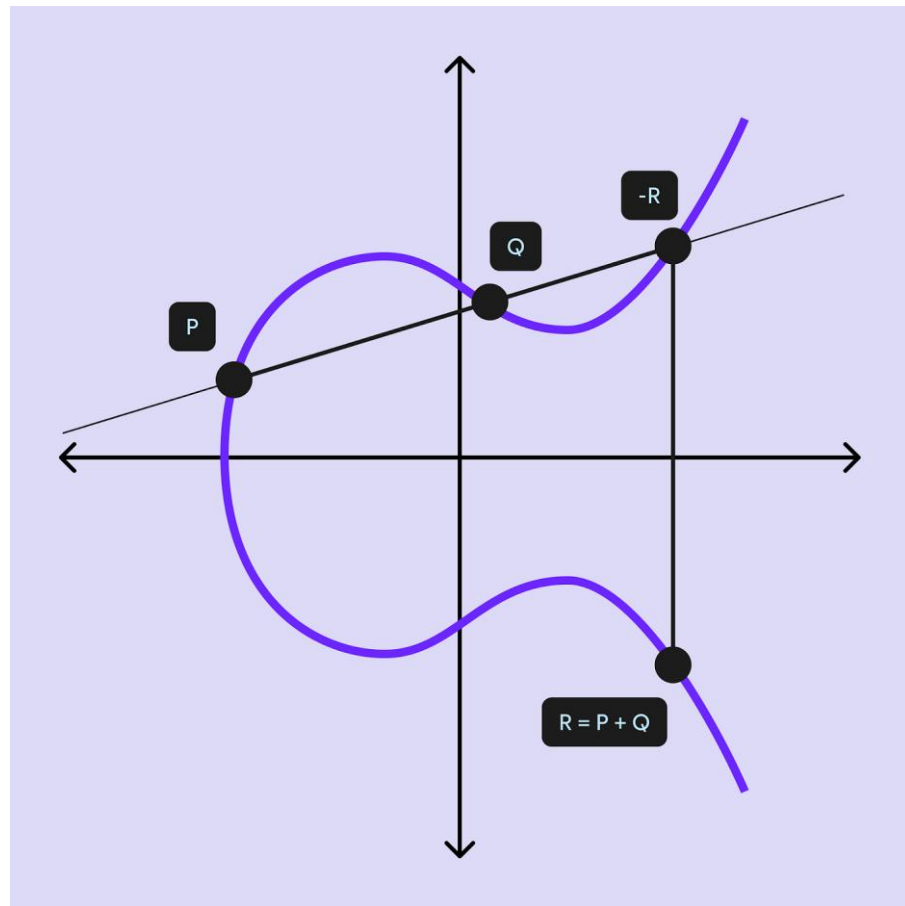
RSA vs ECC

$$y^2 = x^3 + ax + b$$

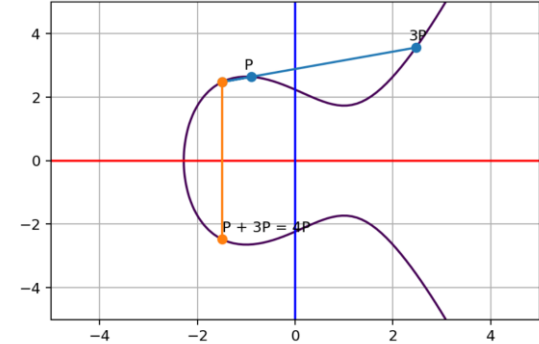
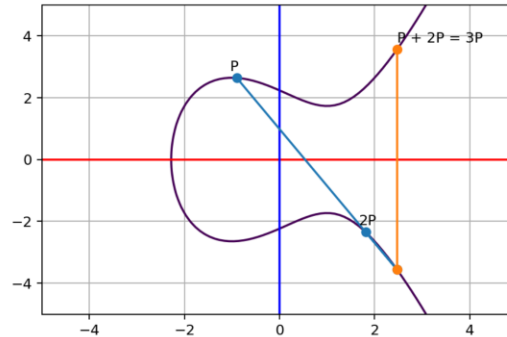
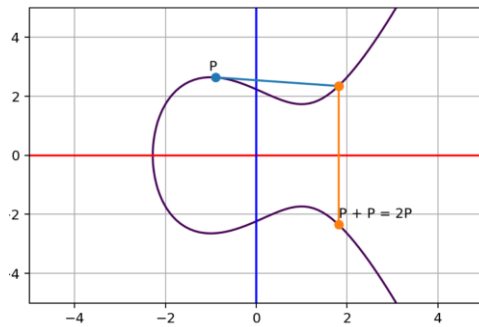
ECC: Elliptic Curve Cryptography

Uma curva elíptica é uma função do tipo:

- $y^2 = x^3 + ax + b$
- A própria **curva elíptica** define um conjunto de pontos possíveis (campo finito ou campo de Galois).
- Sobre esse conjunto, definem-se operações como a **soma de pontos**, usando regras geométricas como as mostradas na figura.

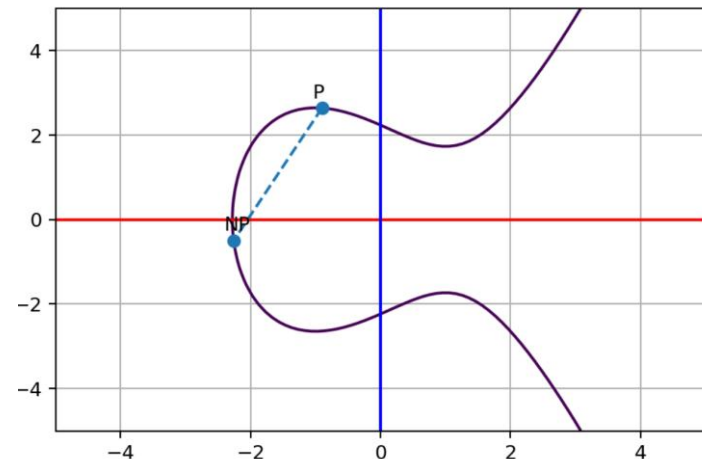


Dobrar e Somar



A curva e o ponto P são fixos e públicos.

- NP é a chave pública
- N é a chave privada.



Na ECC, é fácil calcular $Q = nP$, isto é, somar o ponto P a ele mesmo n vezes.
O difícil é fazer o caminho inverso: receber P e Q e descobrir qual foi o número n usado.

O nome deste problema é logaritmo discreto

Determinar NP se N é conhecido é fácil

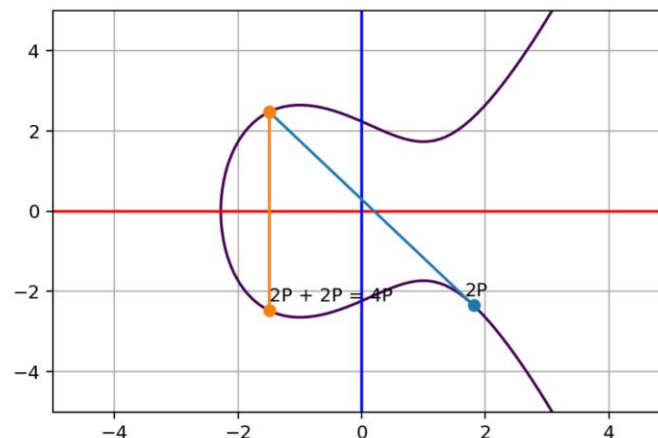
A interseção entre 2 pontos é a soma.
A operação de tangente é dobrar.

Suponha que $N = 227$.

Ele pode ser decomposto em:

$$2^7 P + 2^6 P + 2^5 P + 2^1 P + 2^0 P$$

$$128P + 64P + 32P + 2P + P$$



$$P \rightarrow 2P \rightarrow 4P \rightarrow 8P \rightarrow 16P \rightarrow 32P \rightarrow 64P \rightarrow 128P$$

Isso exige apenas **7 tangentes**.

Depois, para combinar os termos presentes na decomposição binária, você faz **4 intersecções**.

Quiz 7

Algoritmos assimétricos como RSA e ECC só oferece confidencialidade quando as chaves são usadas de qual forma?

- A. Quando a criptografia é feita com a **chave privada** e a descryptografia com a **chave pública**.
- B. Quando a criptografia é feita com a **chave pública** e a descryptografia com a **chave privada**.
- C. Quando a criptografia e a descryptografia são feitas com a **chave privada**.
- D. Quando a criptografia e a descryptografia são feitas com a **chave pública**.
- E. Quando as chaves pública e privada nunca são transmitidas em aberto pela rede.

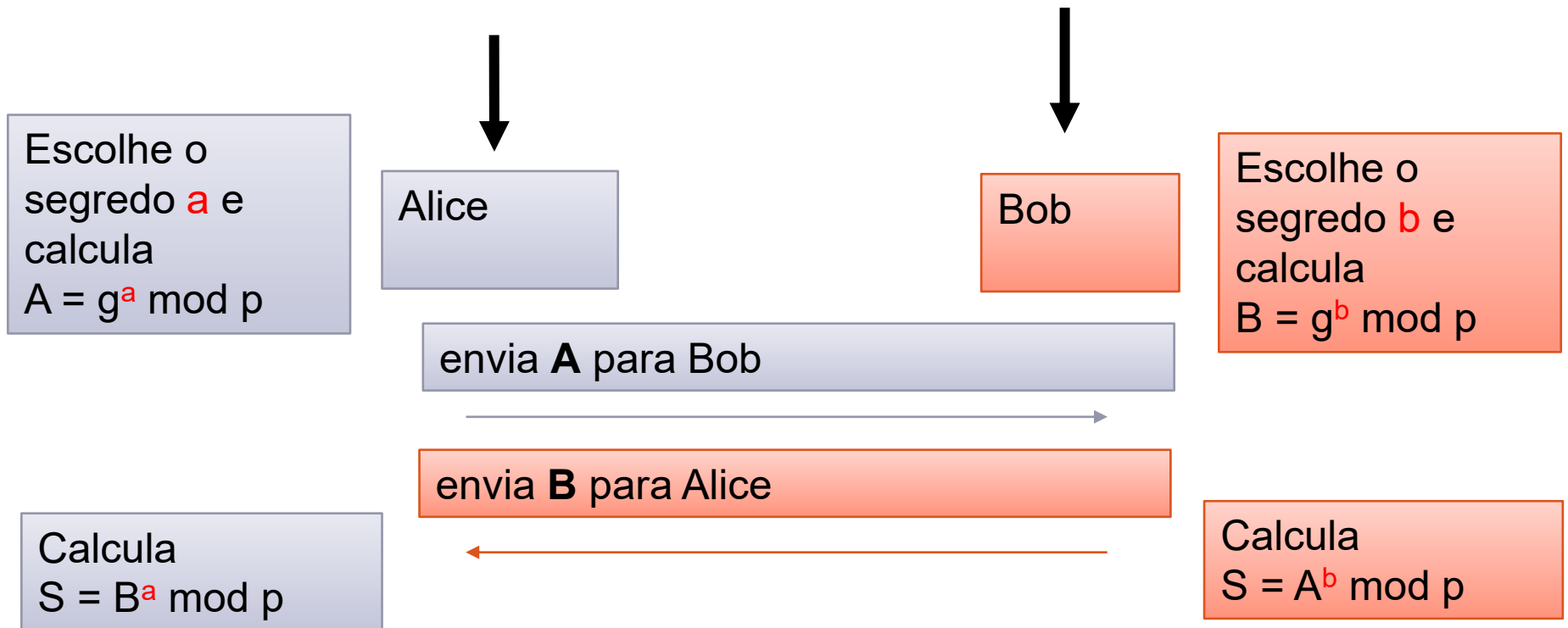
Diffie-Hellman Key Exchange

Whitfield Diffie e **Martin Hellman** introduziram, em **1976**, o algoritmo **Diffie-Hellman Key Exchange**. Nele, os dois lados têm um valor **privado** e um valor **público**, e o objetivo é **combinar um novo segredo compartilhado** pela rede, sem transmitir esse segredo diretamente e sem depender de um segredo previamente compartilhado.



Diffie-Hellman: Variante Básica (modulo)

um número primo $p > 2$ e um número inteiro $g < p$



Diffie-HellMan: Módulo

$p = 23$ e $g = 5$

Escolhe $a=4$
Calcula $5^4 \bmod 23 = 4$

Alice

Bob

Escolhe $b=3$
Calcula $5^3 \bmod 23 = 10$

envia 4 para Bob

envia 10 para Alice

Calcula
 $10^4 \bmod 23 = 18$

Calcula
 $4^3 \bmod 23 = 18$

Diffie-HellMan: ECC

Lembre-se que *
NÃO É UMA MULTIPLICAÇÃO
CONVENCIONAL

Alice e Bob concordam em usar o ponto inicial G

Escolhe **a**
Calcula $A = a * G$

Alice

Bob

Escolhe **b**
Calcula $B = b * G$

envia **A** para Bob

envia **B** para Alice

Calcula
 $S = a * B$

Calcula
 $S = b * A$

$S = a * (b * G)$

$S = b * (a * G)$

DH vs RSA e ECC

No **RSA** e no uso mais comum da **ECC para autenticação**, os dois lados **não fazem a mesma coisa** com as chaves: um lado usa a **chave privada** para assinar, e o outro usa a **chave pública** para verificar. Os papéis são diferentes.

Já no **DH/ECDH**, os dois lados são **equivalentes**: ambos têm seu próprio valor **privado** e seu correspondente valor **público**, e ambos executam o mesmo tipo de operação para chegar ao mesmo segredo compartilhado. Portanto, no DH, os dois participantes têm o mesmo papel; no RSA e na autenticação com ECC, os papéis são distintos.

Grupos Diffie-Hellman

Grupo	Tipo	Tamanho / Curva	Uso atual	
14	MODP	2048 bits	Ainda comum em legado e compatibilidade	
15	MODP	3072 bits	Uso menor; alternativa mais forte que 14	
16	MODP	4096 bits	Mais pesado; uso específico	
19	ECDH	P-256	Muito usado, especialmente em ambientes padronizados	
20	ECDH	P-384	Uso atual quando se quer segurança maior	
21	ECDH	P-521	Uso mais raro	
31	ECDH	X25519	Um dos grupos mais usados hoje	
32	ECDH	X448	Uso atual, mas bem menos comum que X25519	

CRIPTOGRAFIA SIMÉTRICA E ASSIMÉTRICA

FIM

O que são coprimos?

Um número primo é um número divisível apenas por 1 e por ele mesmo.

Coprime é um conceito que se aplica a dois números.

- Coprimos são números primos entre si
- Isto é, não tem divisor em comum (além do 1)

Exemplo: 12 e 15 não são coprimos

- Divisores de 12: 2, 3, 4, 6, 12
- Divisores de 15: 3, 5, 15

Exemplo: 15 e 14 são coprimos

- Divisores de 15: 3, 5, 15
- Divisores de 14: 2, 7, 14